Outbox Pattern

The Outbox Pattern (Why It Matters in Microservices)

You work with Spring, Kafka, microservices — so this is relevant.

The Real Problem

You update the database and publish a Kafka event:

```
orderRepository.save(order);
kafkaTemplate.send("order-created", event);
```

What happens if:

- DB succeeds ✅
- Kafka publish fails ❌

Your system is now inconsistent.

Or worse:

- Kafka succeeds
- DB transaction rolls back

Now other services think the order exists — but it doesn't.

That's a **distributed consistency bug**.

The Outbox Solution

Instead of publishing directly:

1. Save your entity
2. Save an **event record** in an `outbox` table
3. Commit transaction
4. A background process publishes events from the outbox

Everything inside **one DB transaction**.

> "We never send directly to Kafka. We save the event to DB first in the **same transaction** as the business data, then a scheduler sends it to Kafka. This guarantees **no message is ever lost**."

What It Looks Like

Step 1 — Inside your service

```
@Transactional
public void createOrder(CreateOrderRequest request) {
    Order order = orderRepository.save(...);

```

```
    OutboxEvent event = new OutboxEvent(
        "ORDER_CREATED",
        objectMapper.writeValueAsString(order)
    );

    outboxRepository.save(event);
}
```

Now:

- Either both Order + OutboxEvent are saved
- Or nothing is saved

No inconsistency.

Step 2 — Publisher Process

We already saved something in a table called:

```
outbox_event
```

It contains rows like:

id	type	payload	published
1	ORDER_CREATED	{...json...}	false

Now we need something that:

1. Looks at rows where `published = false`
2. Sends them to Kafka
3. Marks them as `published = true`

That's it.

Super Simple Code Version

```
@Scheduled(fixedDelay = 5000) // every 5 seconds
public void publishEvents() {

    // Get events not yet sent
    List<OutboxEvent> events =
outboxRepository.findByPublishedFalse();

    for (OutboxEvent event : events) {

        // Send to Kafka
        kafkaTemplate.send("order-topic", event.getPayload());

        // Mark as sent
        event.setPublished(true);
        outboxRepository.save(event);
    }
}
```

Why Every 5 Seconds?

Because we don't want to send inside the same transaction as the order.

We let:

- Transaction finish
- Data be safely stored
- Then publish later

Proper Safe Version

If Kafka throws exception and you don't handle it:

The whole scheduler might stop depending on configuration.

So you should do this instead:

```
@Scheduled(fixedDelay = 5000)
public void publishEvents() {

    List<OutboxEvent> events =
outboxRepository.findByPublishedFalse();

    for (OutboxEvent event : events) {
        try {
            kafkaTemplate.send("order-topic",
event.getPayload()).get(); // wait for ack

            event.setPublished(true);
            outboxRepository.save(event);

        } catch (Exception ex) {
            // Do NOT mark as published
            // Log error
            log.error("Failed to publish event {}", event.getId(),
ex);
        }
    }
}
```

Why `.get()` ?

Because `send()` is async.

Without `.get()`:

- You may mark event as published
- Even if Kafka later fails

That would be a bug.

So Where Is Retry?

Retry =
"Keep selecting events where `published = false`"

No complex retry loop needed.

The database state controls retry.